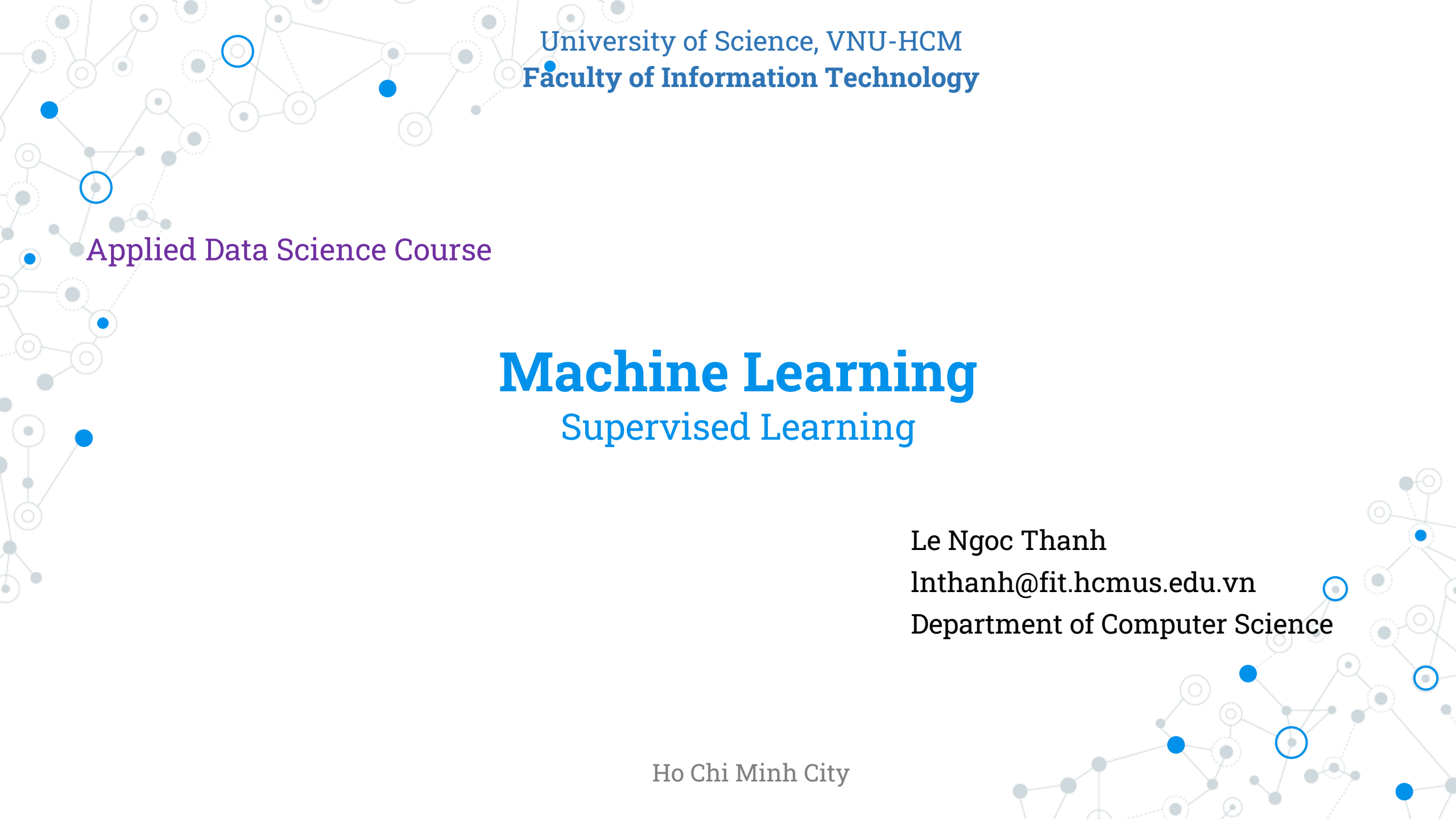




University of Science, VNU-HCM
Faculty of Information Technology

Applied Data Science Course

Machine Learning

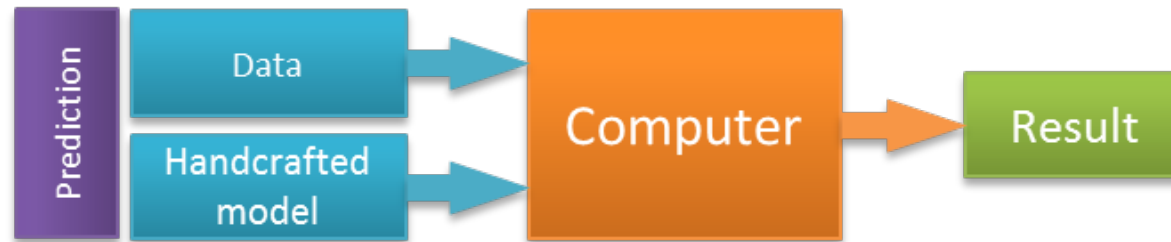
Supervised Learning

Le Ngoc Thanh
lnthanh@fit.hcmus.edu.vn
Department of Computer Science

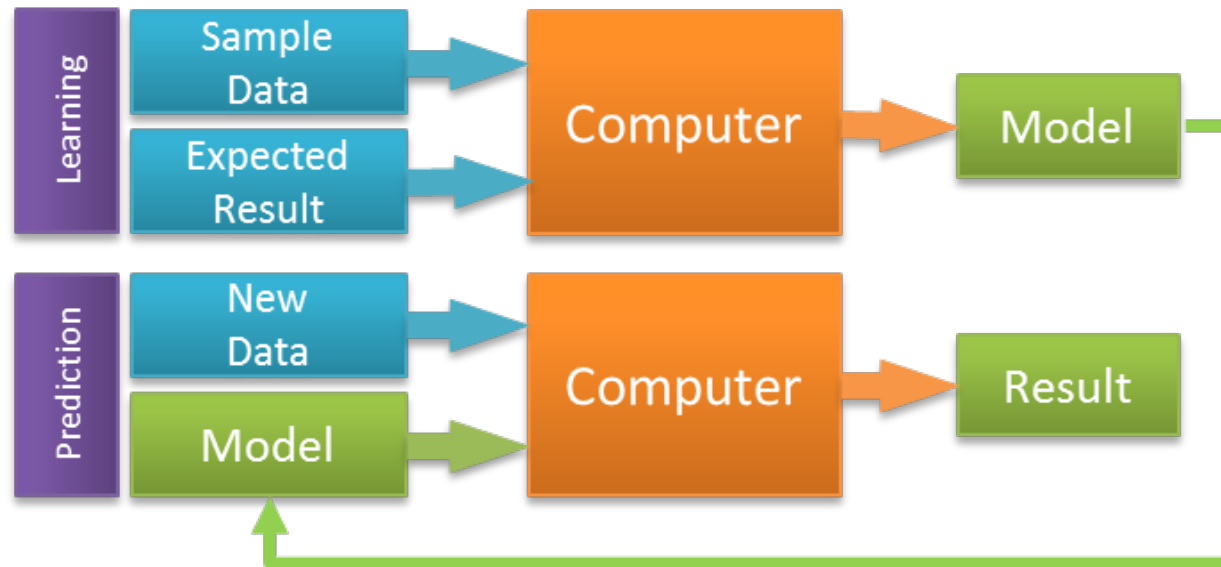
Ho Chi Minh City

Traditional Programming vs Machine Learning

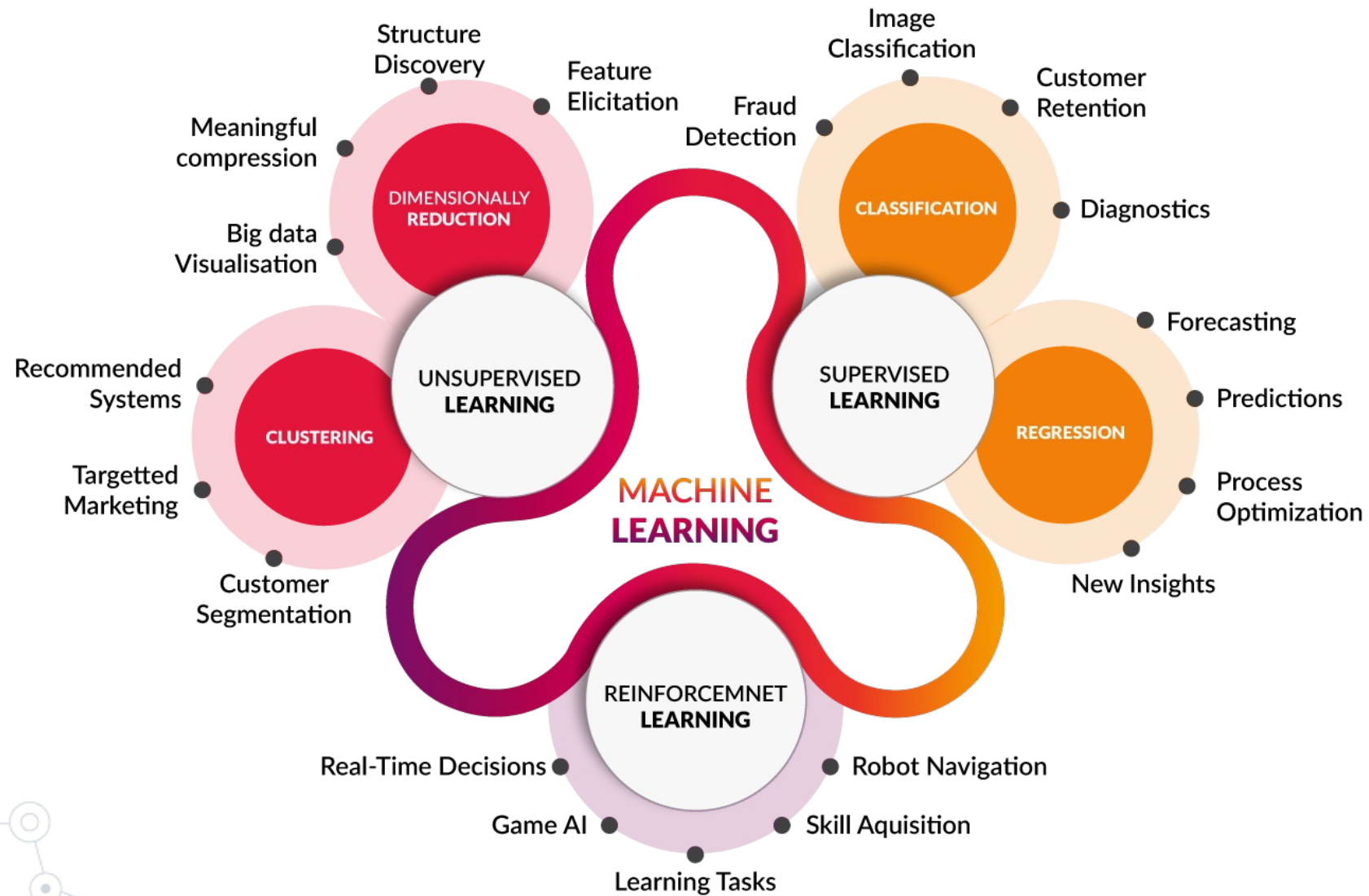
Traditional modeling:



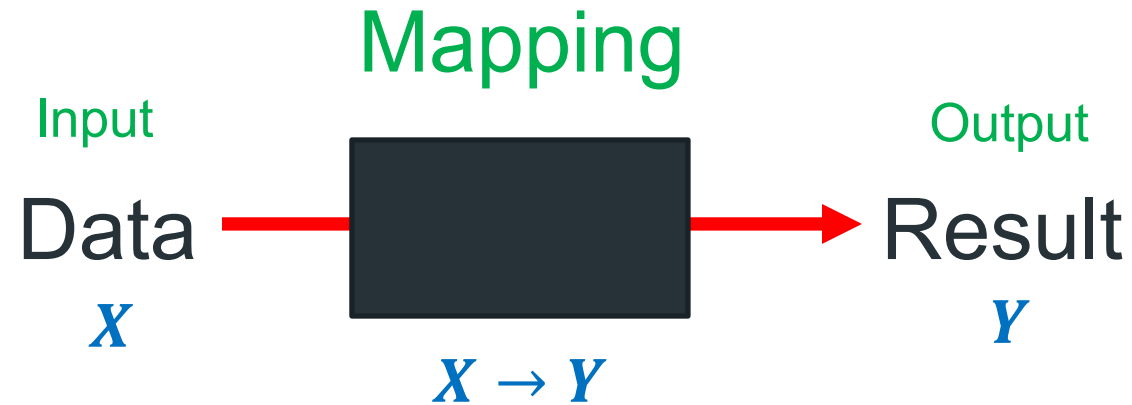
Machine Learning:



ML Tasks



ML Architecture: Mapping



Learning = Identify mapping

How to identify the mapping

- ◎ Hypothesis
- ◎ Hypothesis
- ◎ Hypothesis
- ◎ ... and
- ◎ Hypothesis

You are here



Hypothesis Space

Hypothesis 1

Hypothesis 2

Hypothesis 3

Hypothesis 4

You want here

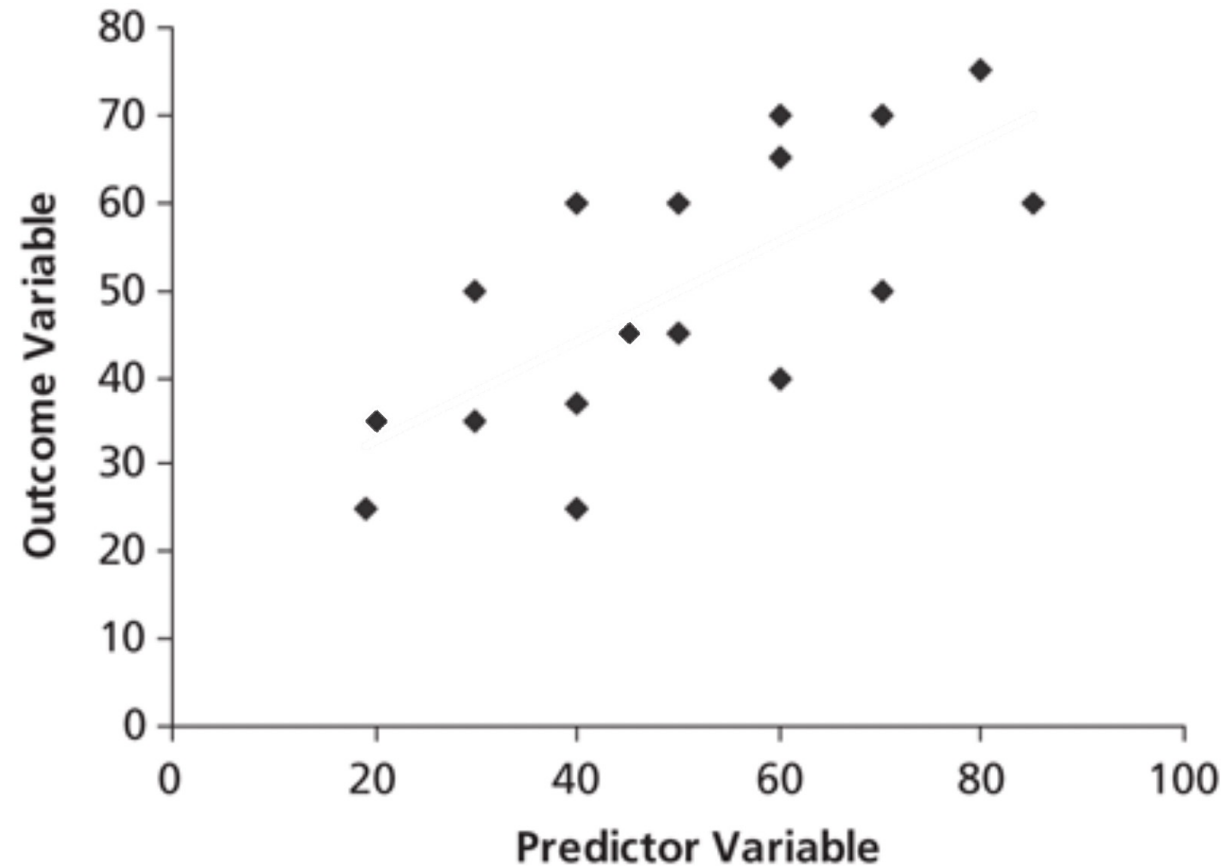


Best friend's hypothesis

Examples about hypothesis

◎ Hypothesis for regression

Observed Data
(Training data)
 $(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})$



Examples about hypothesis

◎ Hypothesis for regression

Observed Data
(Training data)

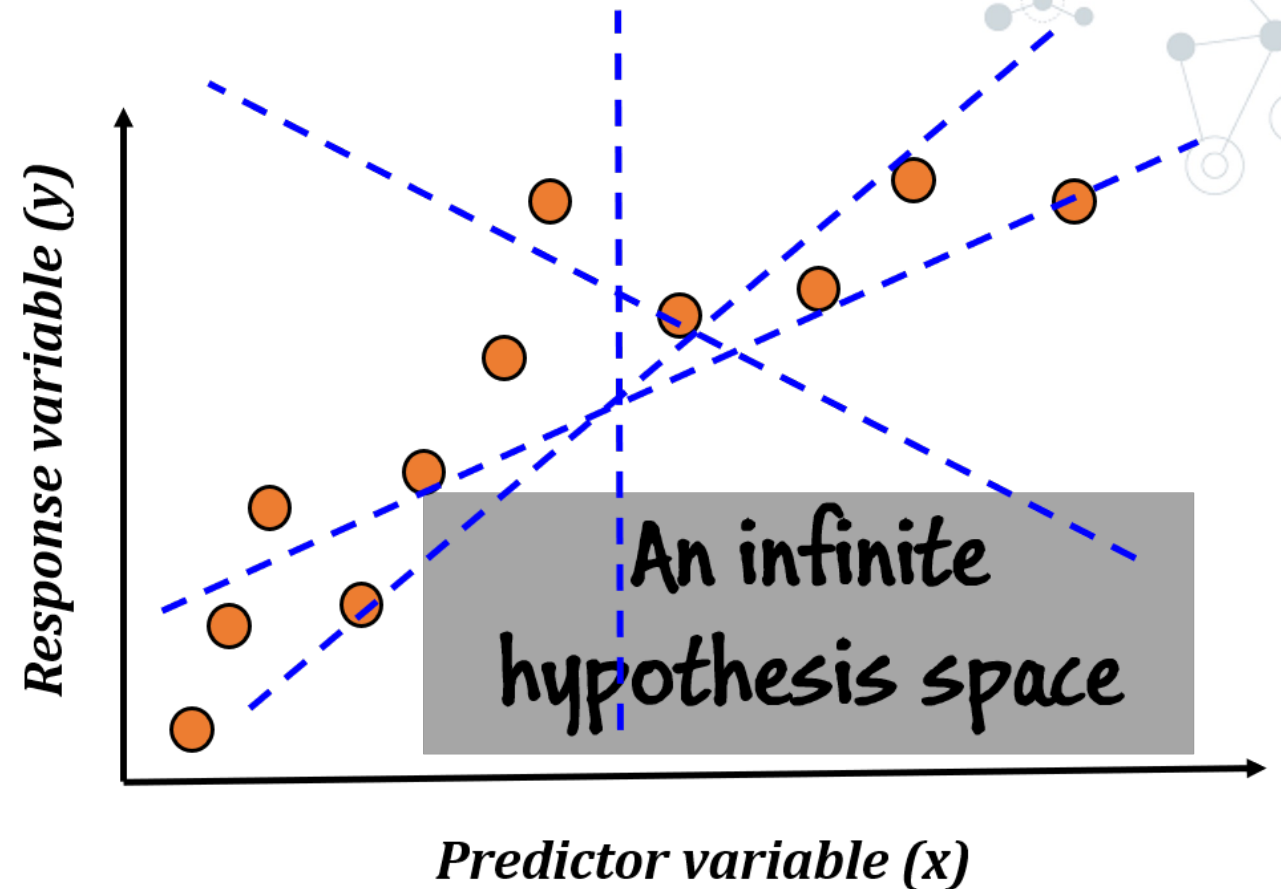
$$(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})$$

Hypothesis

Linear function with
different parameters

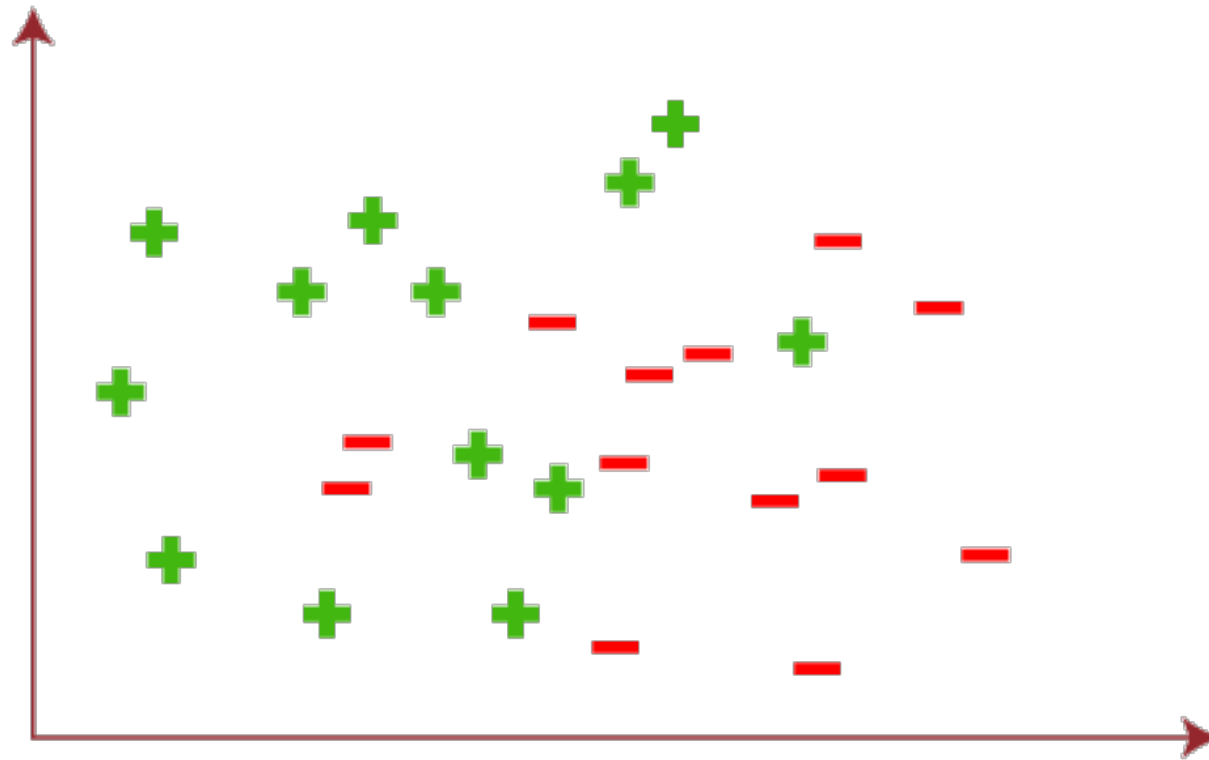
$$h_w(x)$$

$$= w_0 + w_1 x_1 + w_2 x_2 + \dots$$



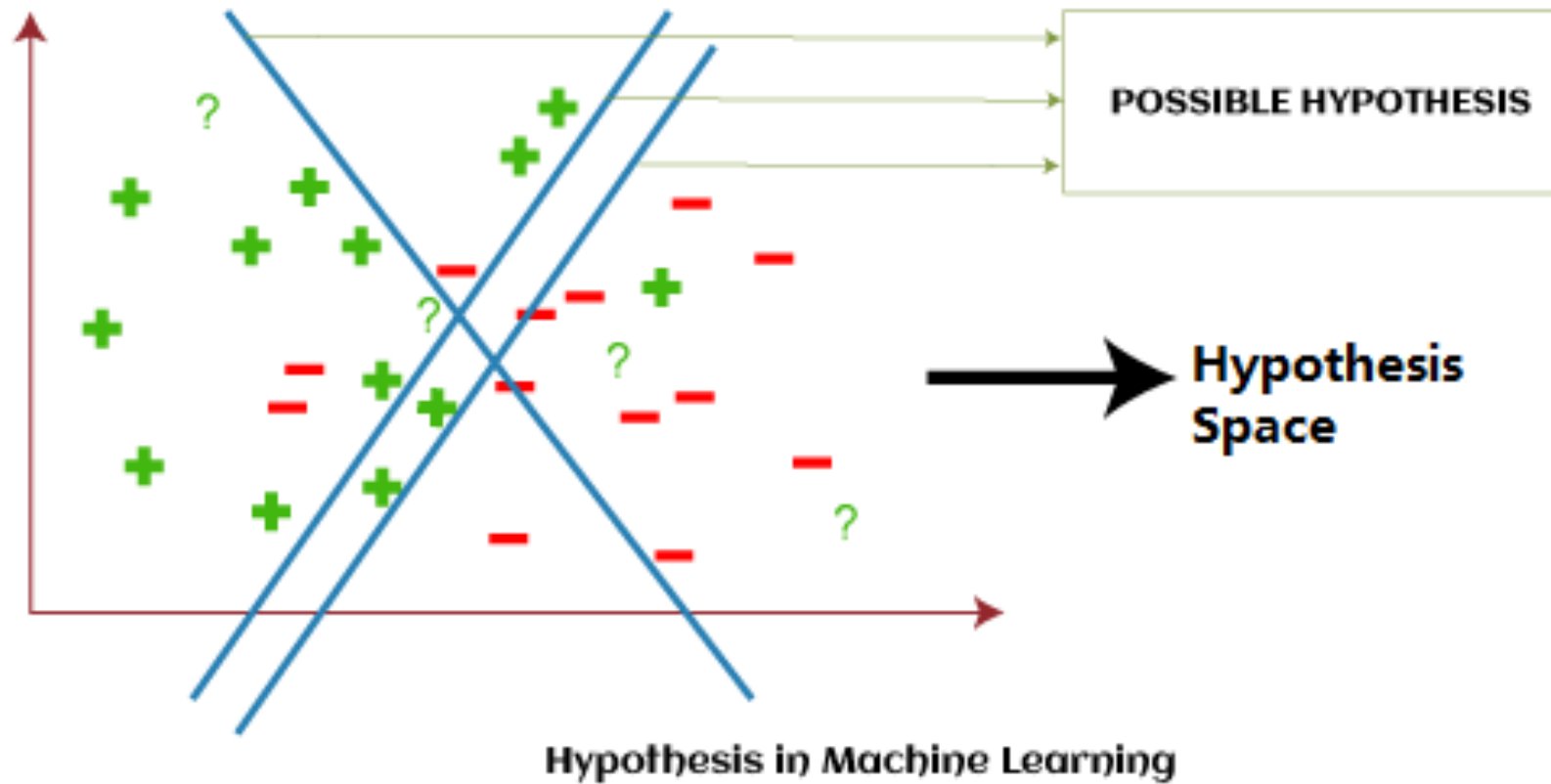
Examples about hypothesis

◎ Hypothesis for classification



Examples about hypothesis

◎ Hypothesis for classification



After hypothesis

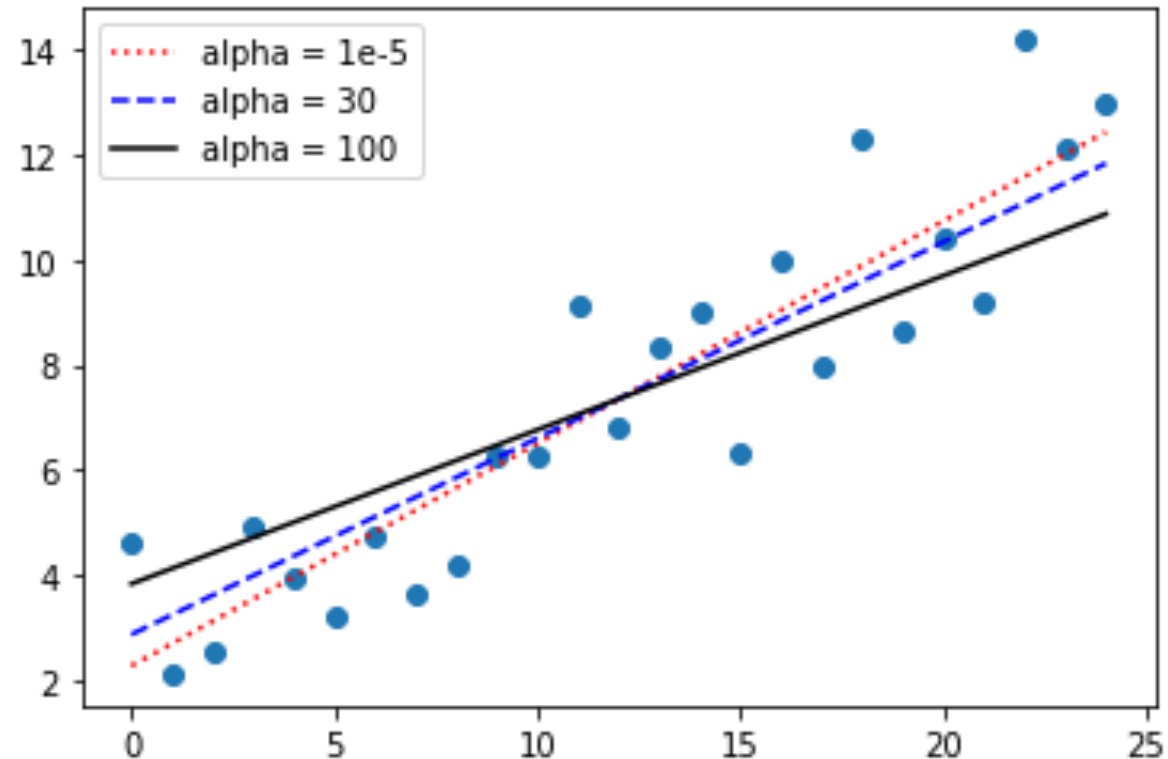
- ◎ To choose the suitable hypothesis, we need to **define** the **loss function**.

$$\mathcal{L}(y - \hat{y}) = \sum_{i=1}^n (y - \hat{y}_i)^2$$

Machine learning = iterative procedure to find a minimum of loss for the given data.

Loss Function Design

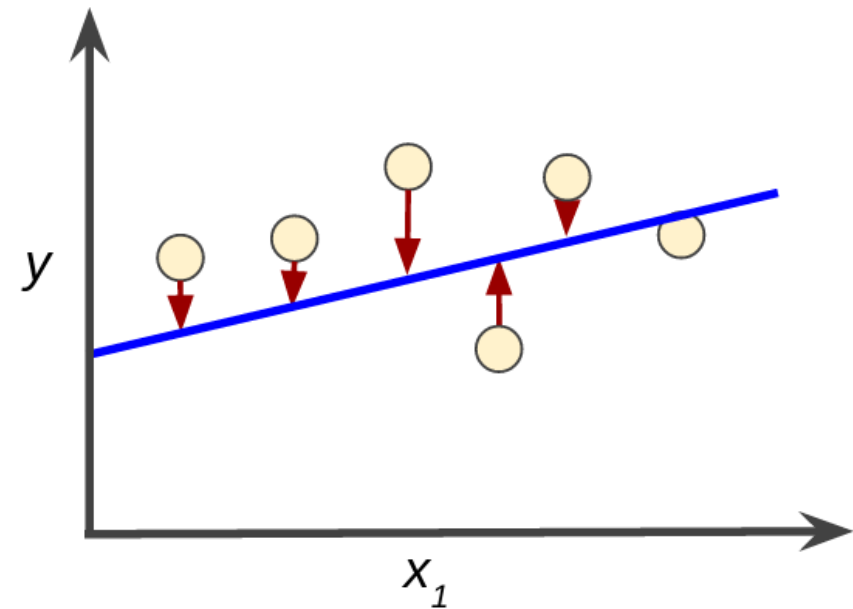
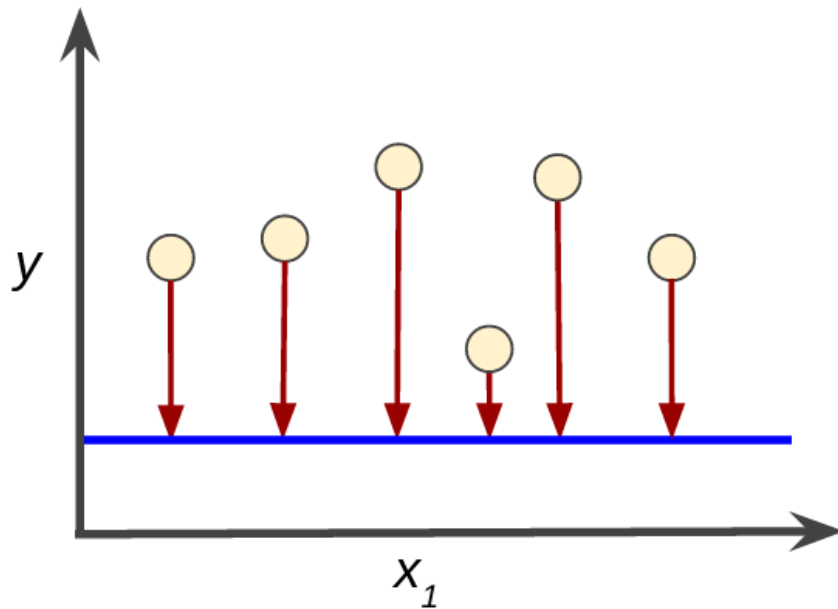
◎ For regression, how to design loss function?



Which is the best linear function?

Loss Function Design

◎ For regression, how to design loss function?



$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Common loss functions

◎ For regression,

- **Mean square error** (quadratic loss, L_2 loss):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Mean absolute error** (L_1 loss):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean bias error:**

$$MBE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i$$

Common loss functions

◎ For classification,

- **Binary cross entropy** (negative log likelihood):

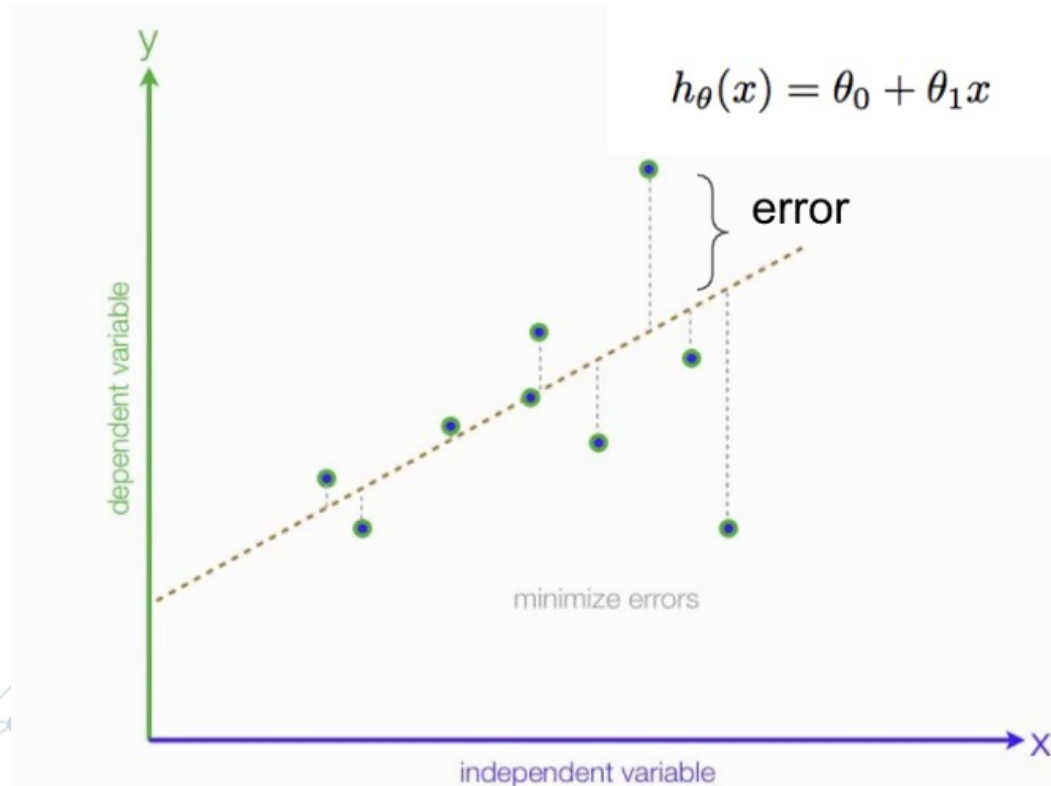
$$CrossEntropyLoss = -\frac{1}{n} \sum_{i=1}^n y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

- **Hinge Loss** (multi class SVM loss):

$$SVMLoss = \sum_{j \neq y_i} \max(0, s_j - s_{y_j} + 1)$$

After loss function design

- ◎ We are looking for **what parameters to produce the lowest loss rate** for given dataset, so we need **the process to optimize the function** (fitting).



Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$

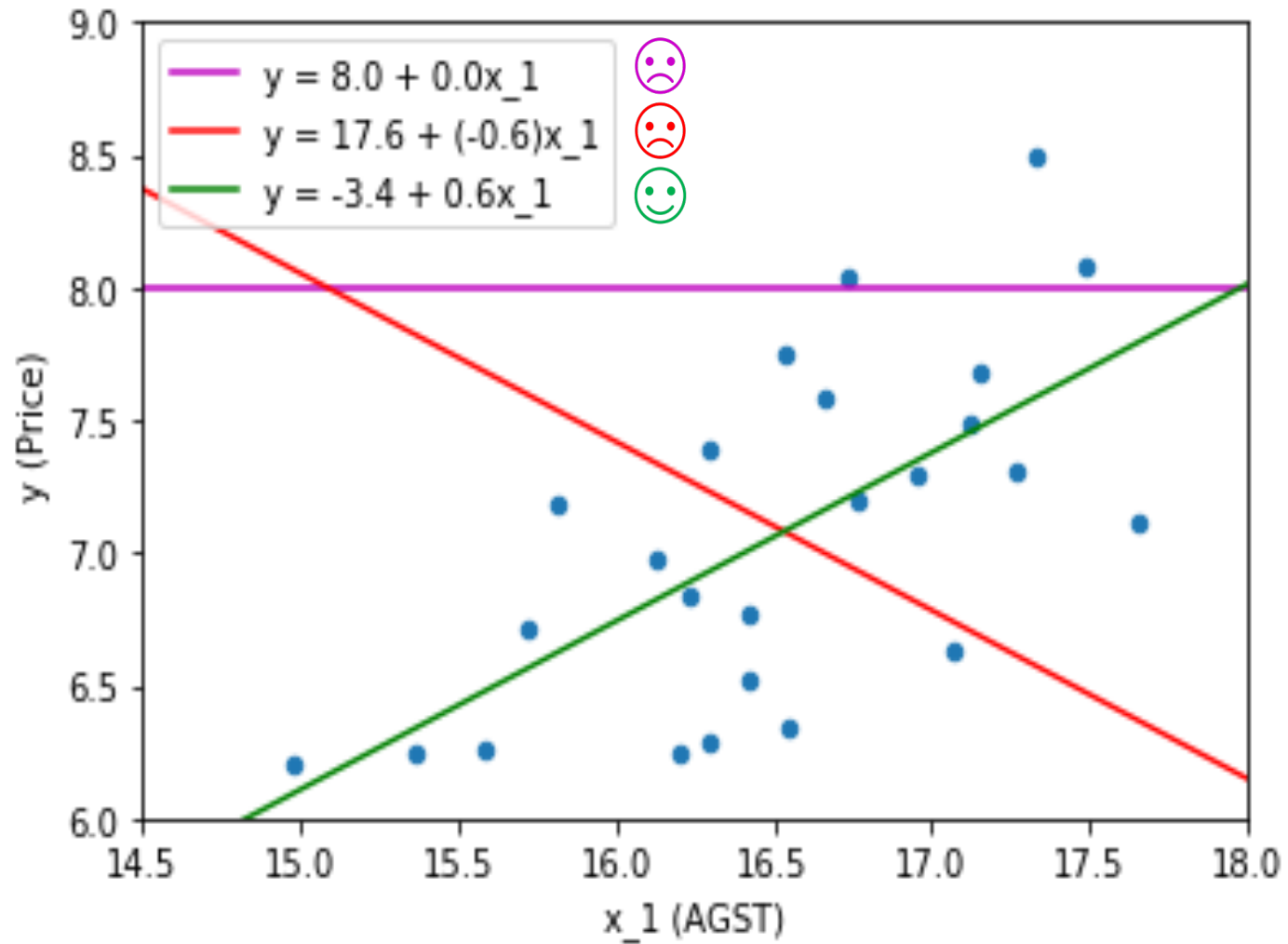
Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal:

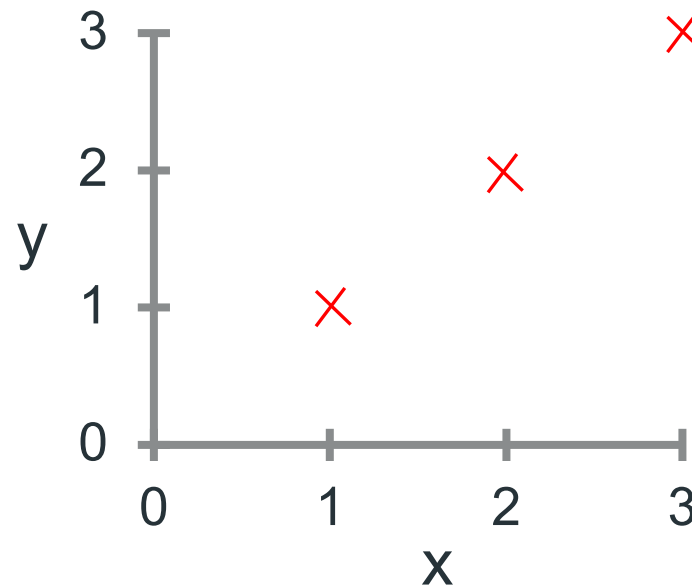
$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$$

Optimization



How to optimize?

© **Method 1** (brute force): **Trial and error**



$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

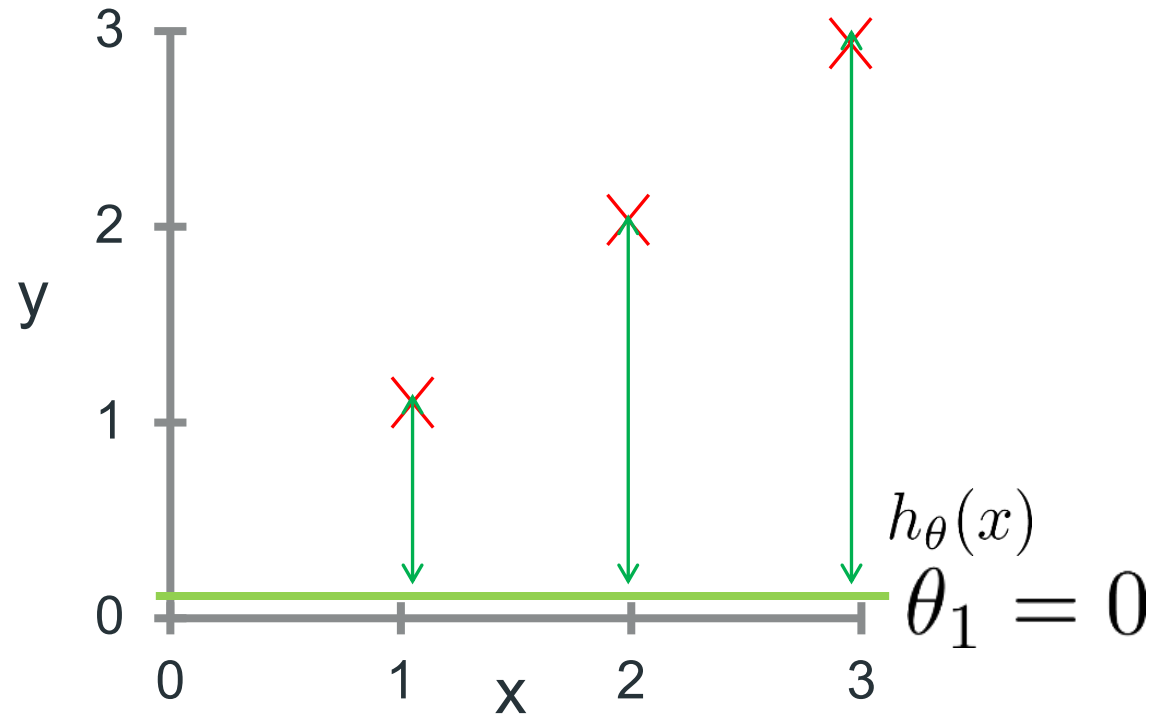
How to optimize?

◎ Method 1 (brute force): Trial and error

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$\theta_0 = 0, \theta_1 = 0$$

↓
 $L = 2.3$



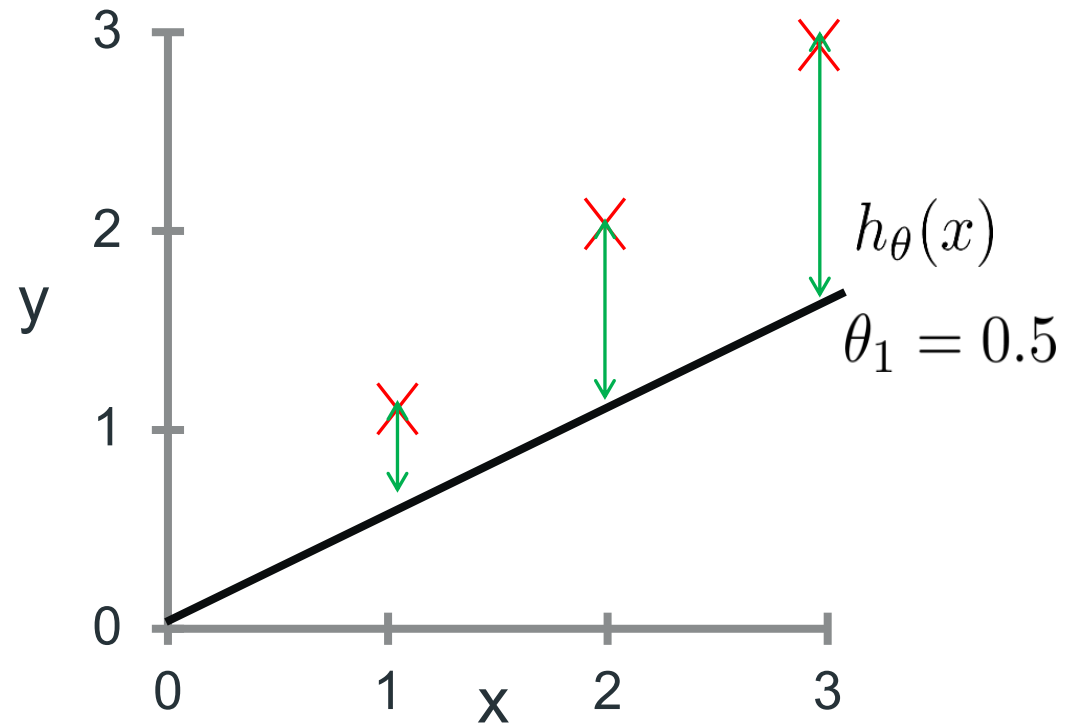
How to optimize?

◎ Method 1 (brute force): Trial and error

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$\theta_0 = 0, \theta_1 = 0.5$$

$$L = 0.58$$



How to optimize?

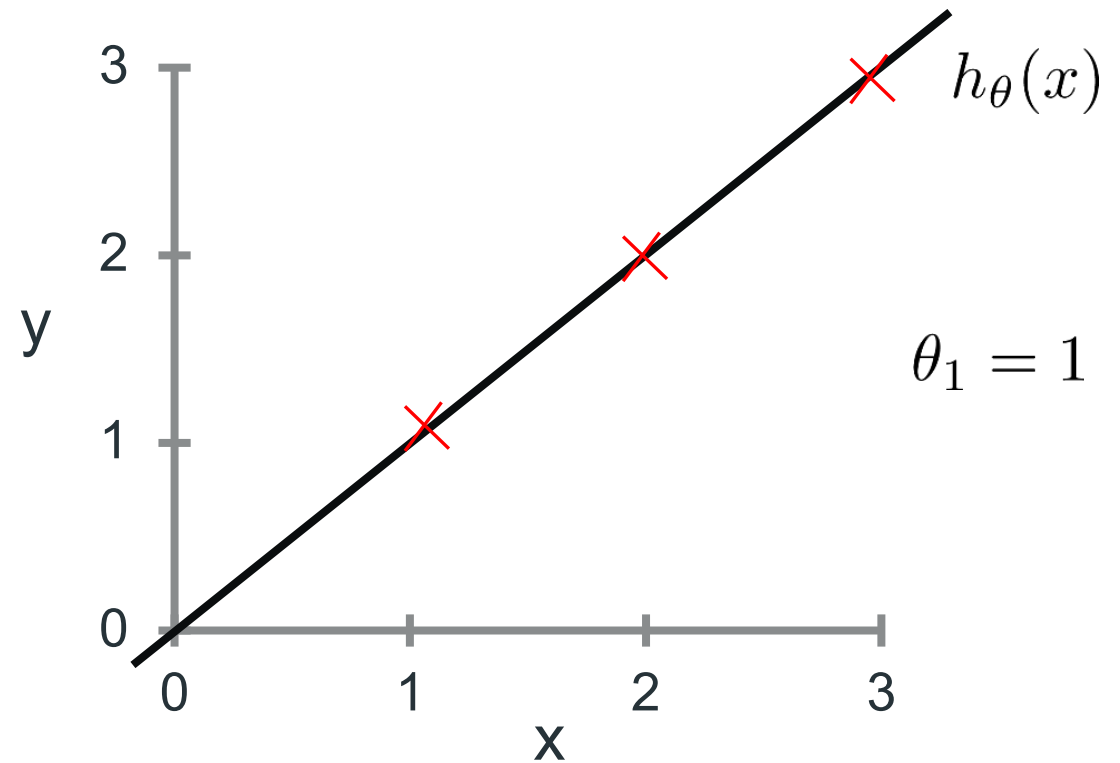
◎ Method 1 (brute force): Trial and error

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$\theta_0 = 0, \theta_1 = 1$$



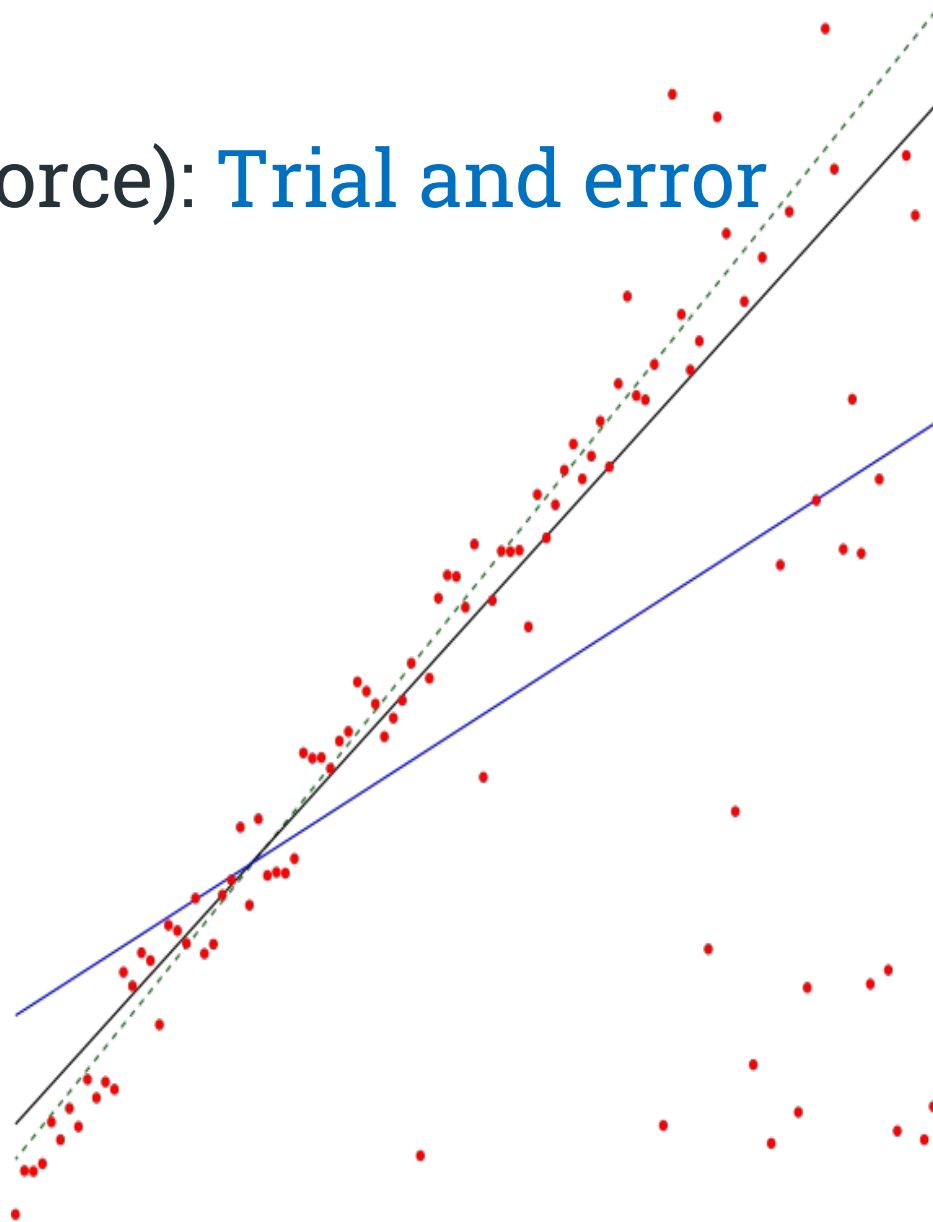
$$L = 0$$



How to optimize?

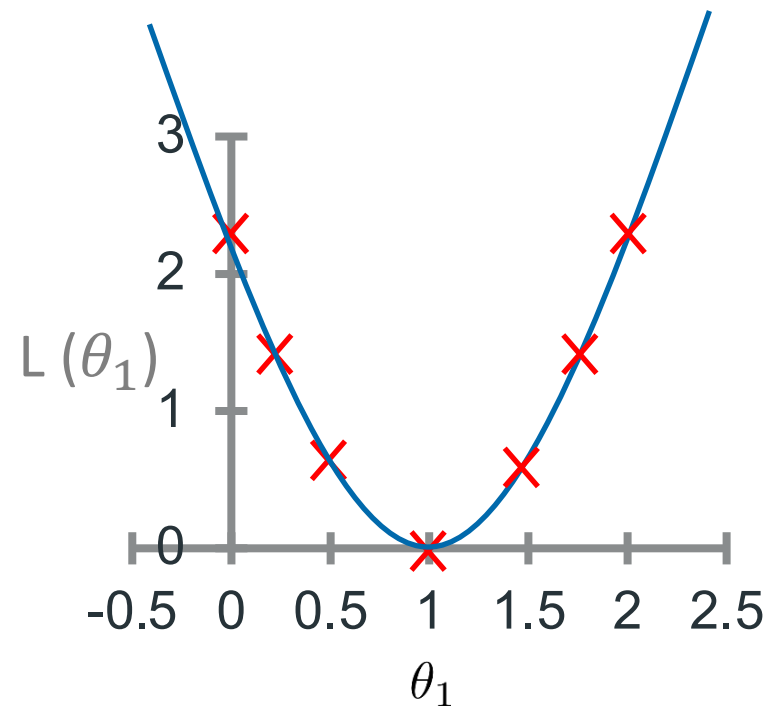
◎ Method 1 (brute force): Trial and error

- Advantage?
- Disadvantage?



How to optimize?

- ◎ **Method 2** (derivative): derivatives to find the extremum point.

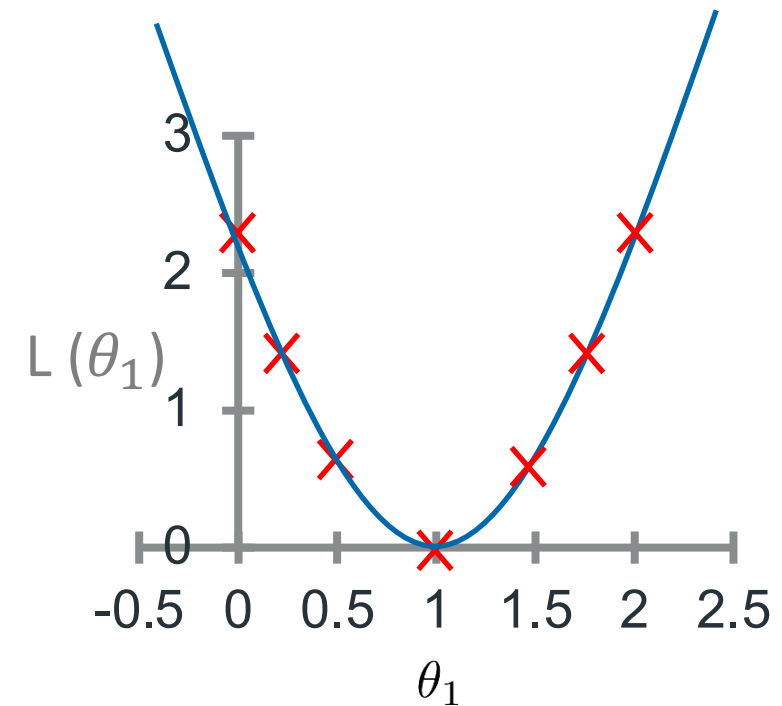


$$L'(\theta) = 0$$

How to optimize?

◎ **Method 2** (derivative): derivatives to find the extremum point.

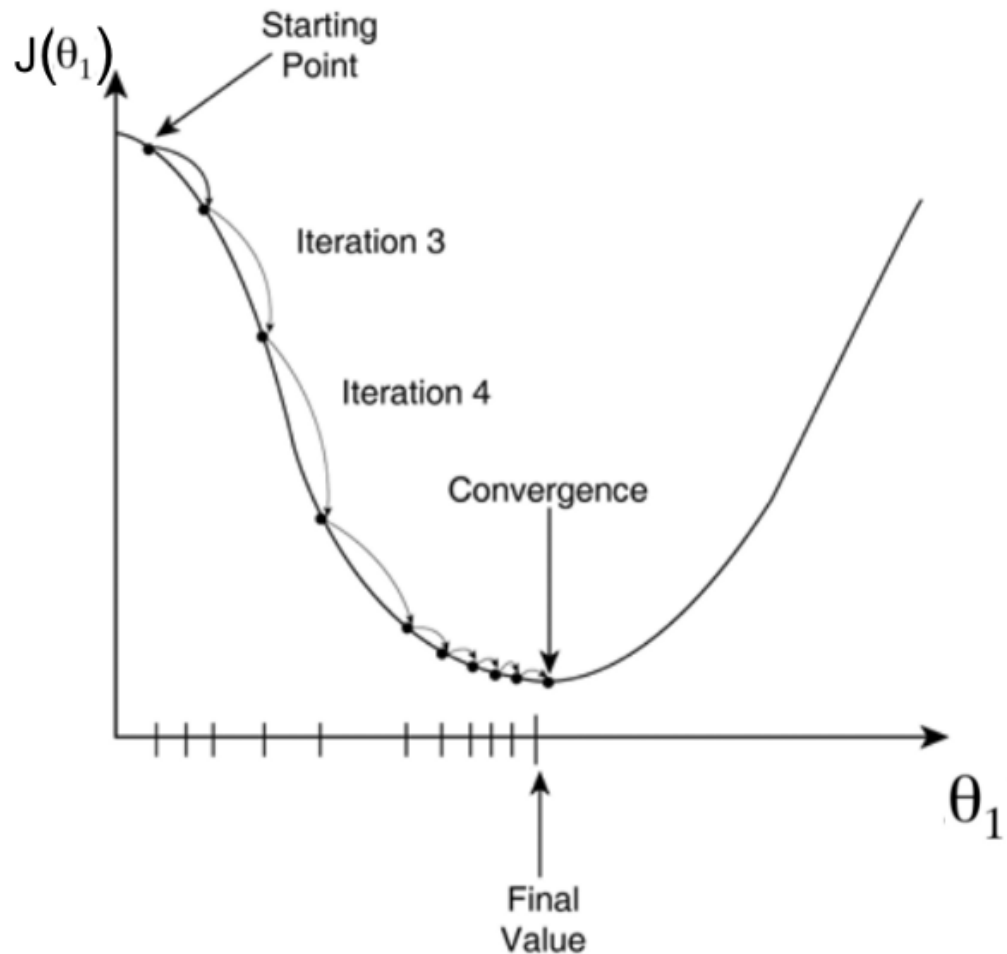
- Advantage?
- Disadvantage?



$$L'(\theta) = 0$$

How to optimize?

Method 3: gradient descent.



Cost Function – “One Half Mean Squared Error”:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Objective:

$$\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

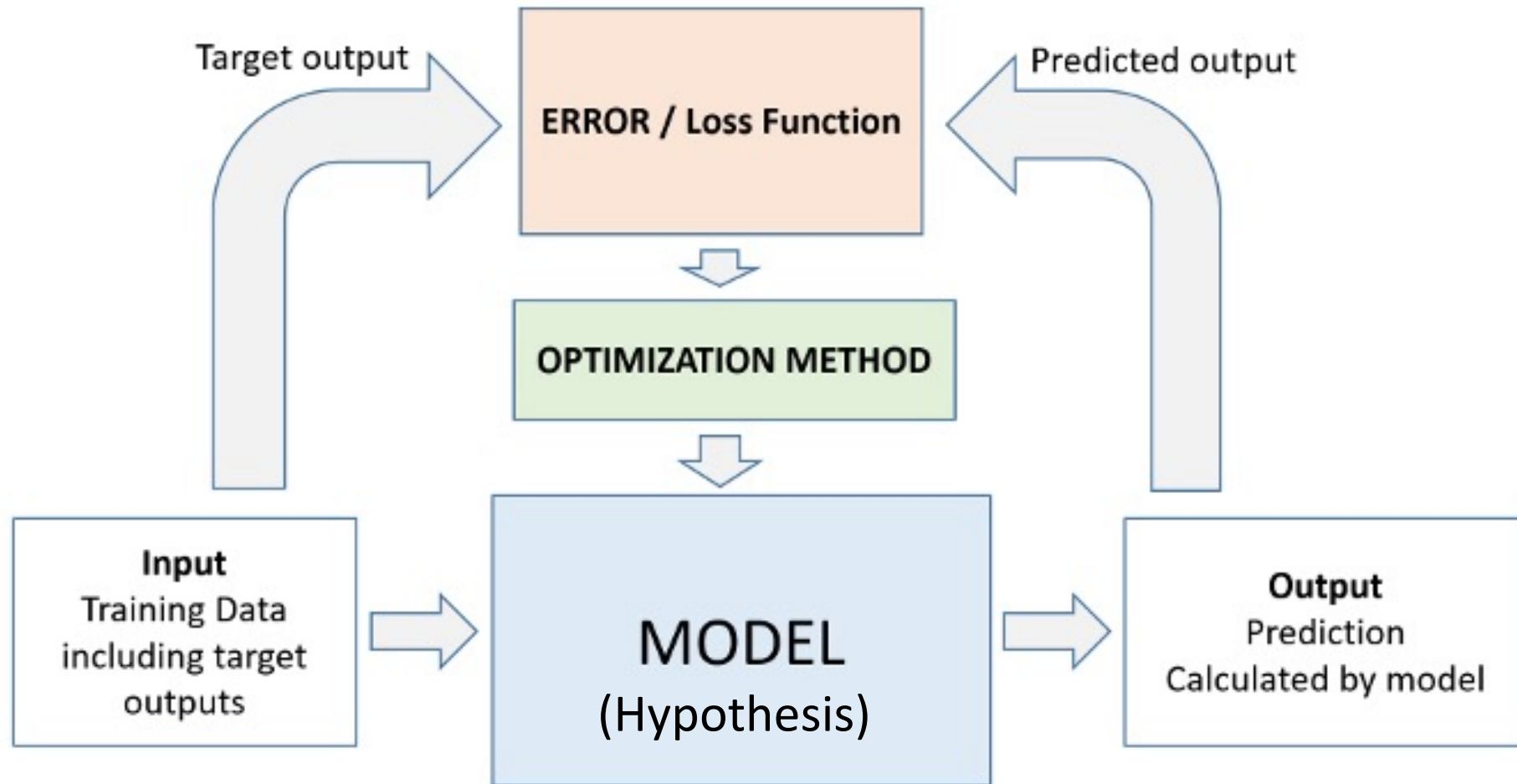
Derivatives:

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

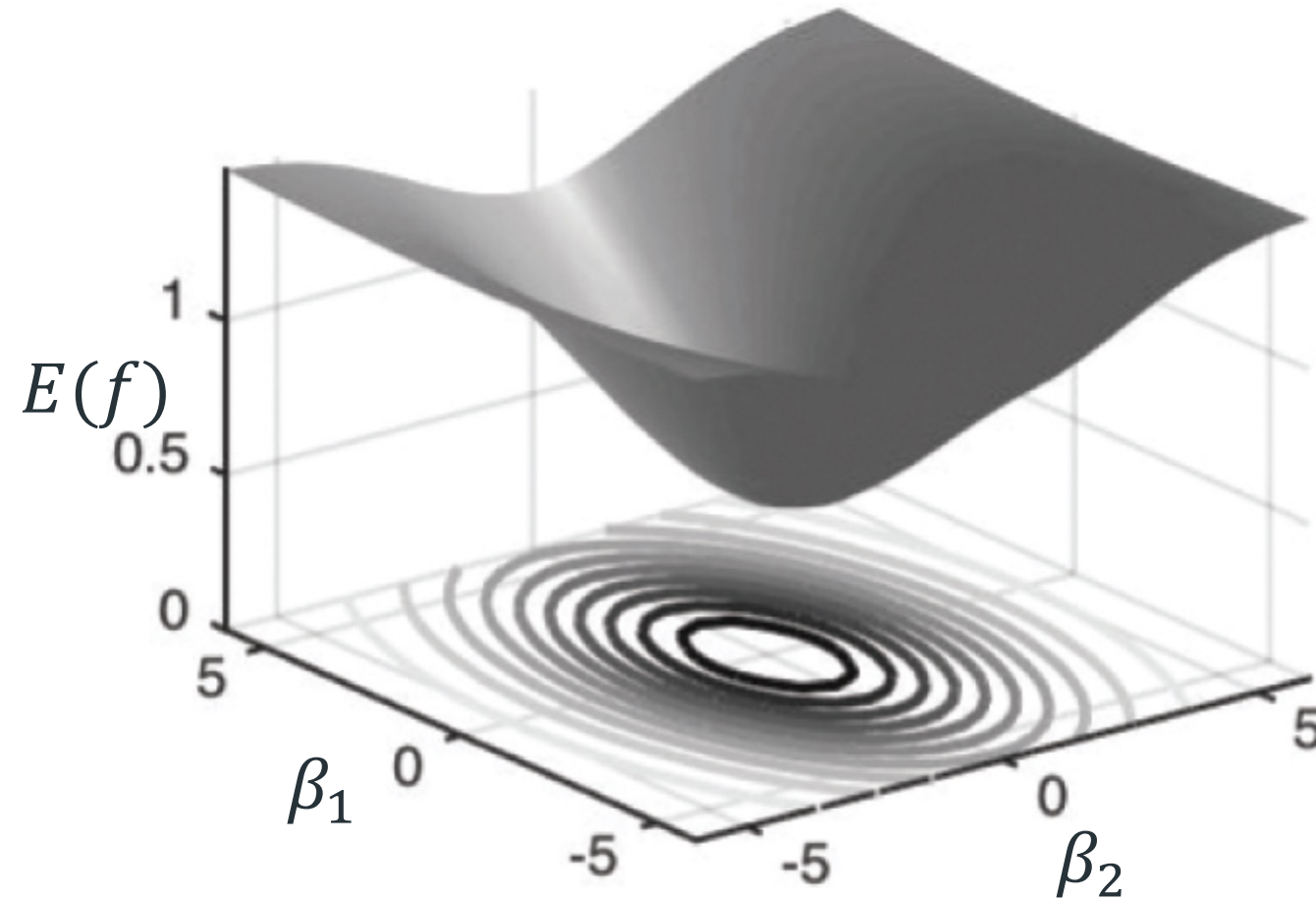
repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)
}

General model learning architecture



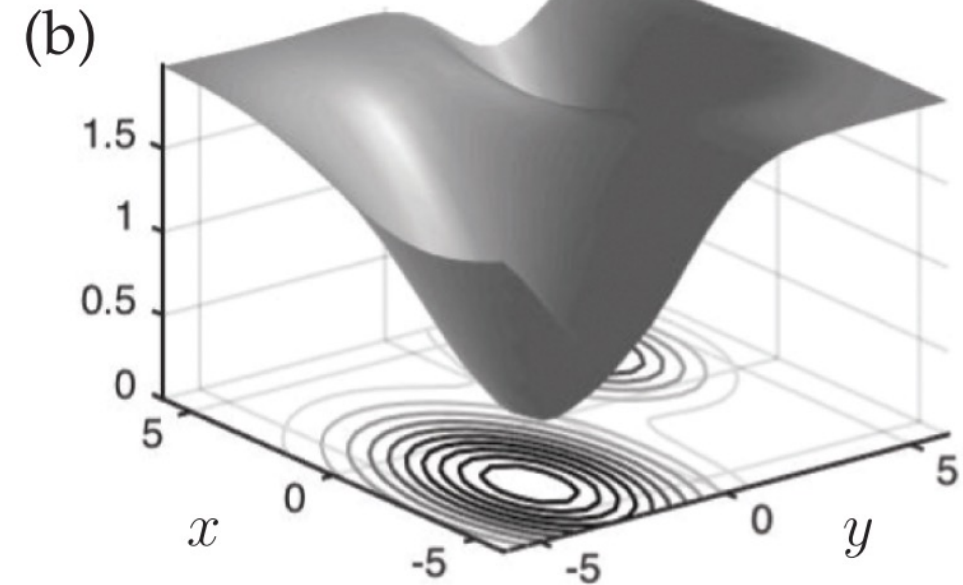
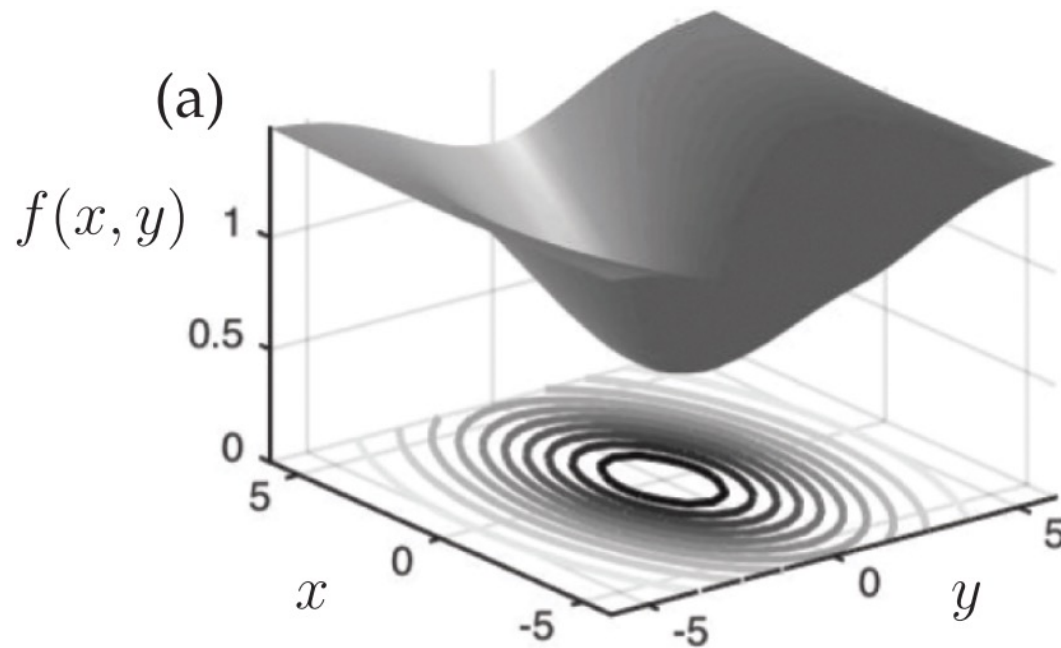
Linear regression

- ◎ The loss function is **MSE** (mean-squares error)



Non-linear vs linear function

◎ Convex function and nonconvex function

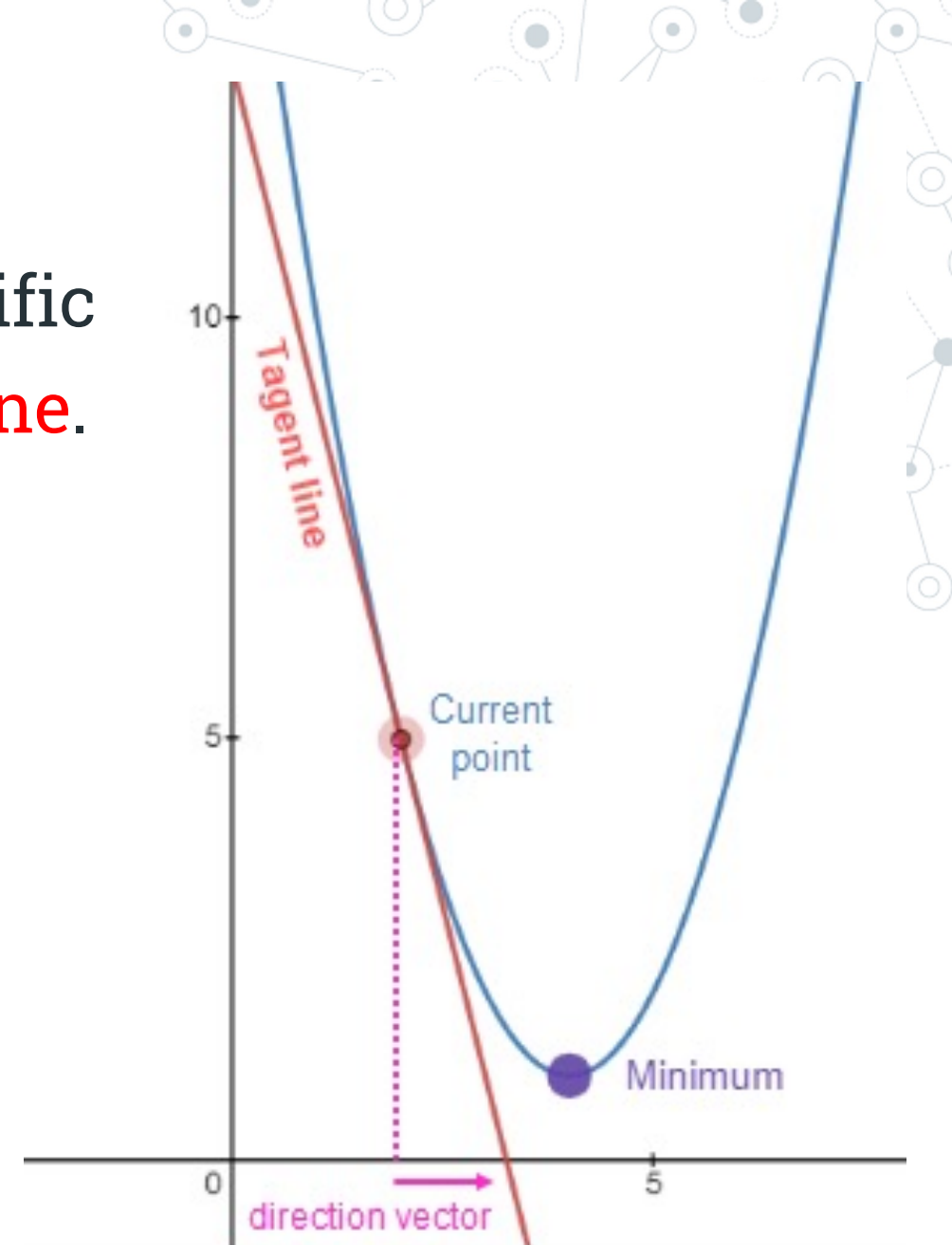


Direction vector

- ◎ The **derivative** of a function at a specific point gives the **slope of the tangent line**.

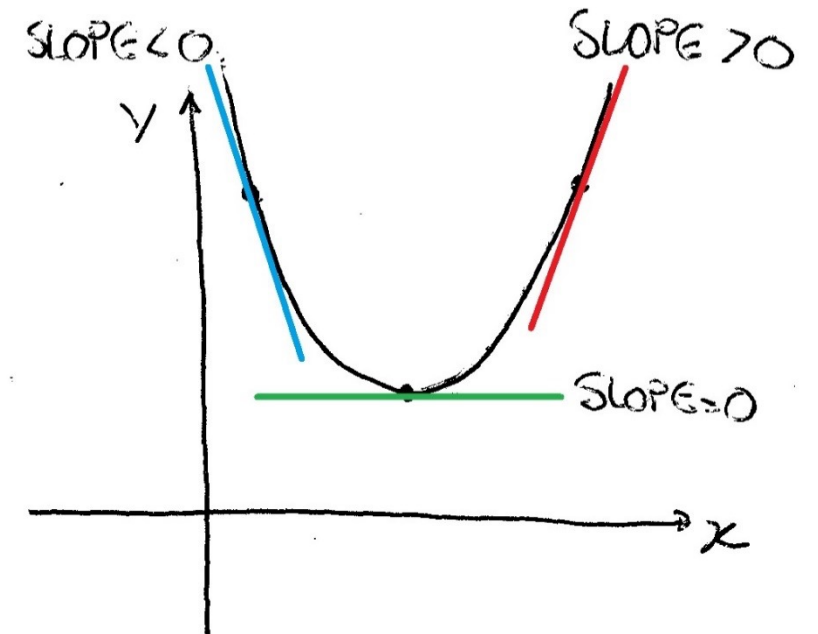
$$f'(x) = \lim_{(x_1 - x_0) \rightarrow 0} \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

- ◎ Why is **the tangent line** considered as a **direction vector**?



Direction vector

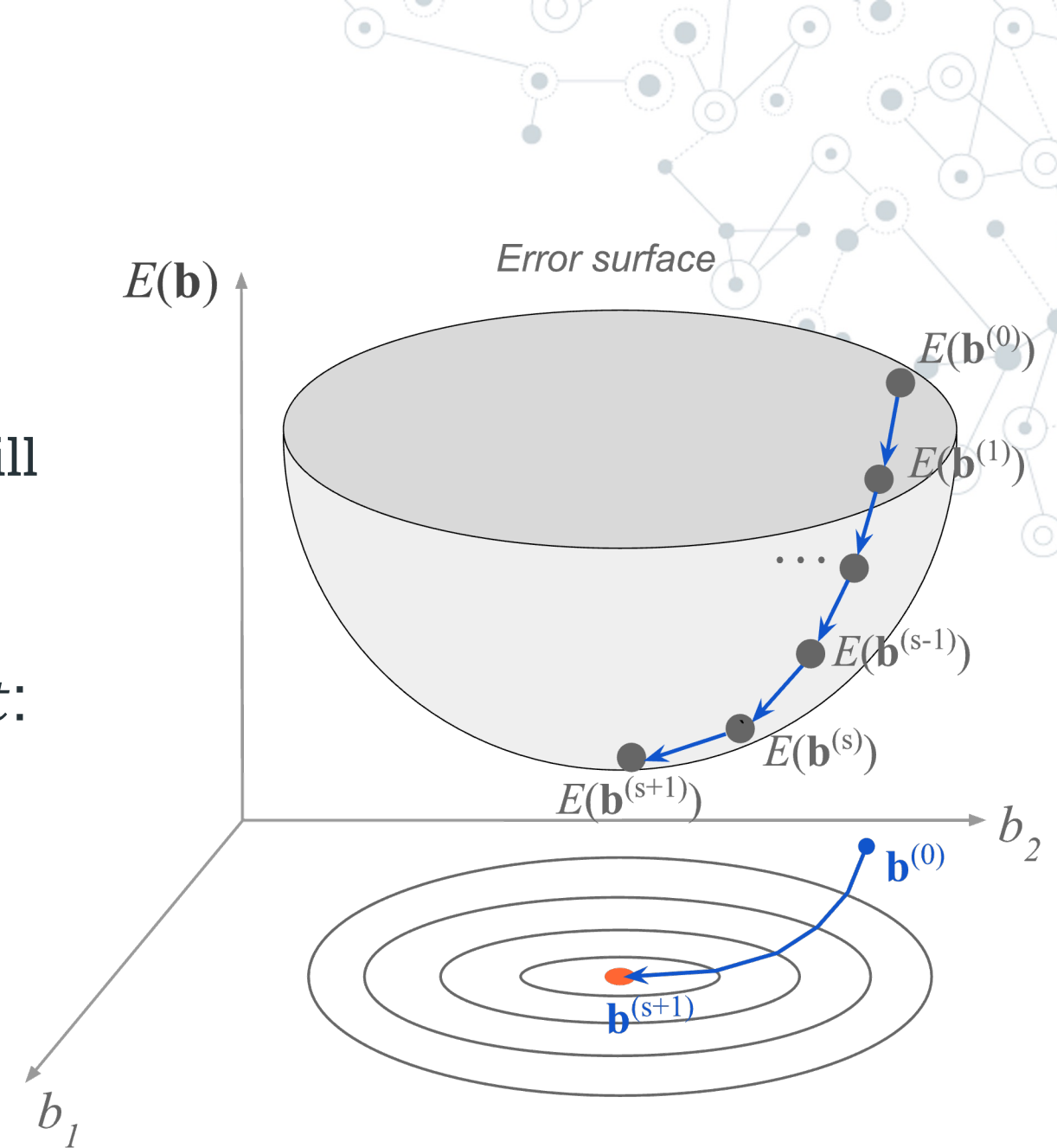
- ◎ If the slope of the tangent line = 0, we will have the **extremum point**.
- ◎ Slope < 0: when x increase, f(x) decrease.
- ◎ Slope > 0: when x increase, f(x) increase.
⇒ **Direction we want is the opposite of slope.**



Gradient Descent

- ◎ As we **update**, we want the value of $f(x, y)$ to **decrease**.
 - When it stops decreasing, (x_0, y_0) will have arrived at the position giving the minimum value of $f(x, y)$.
- ◎ The next position at time step t :

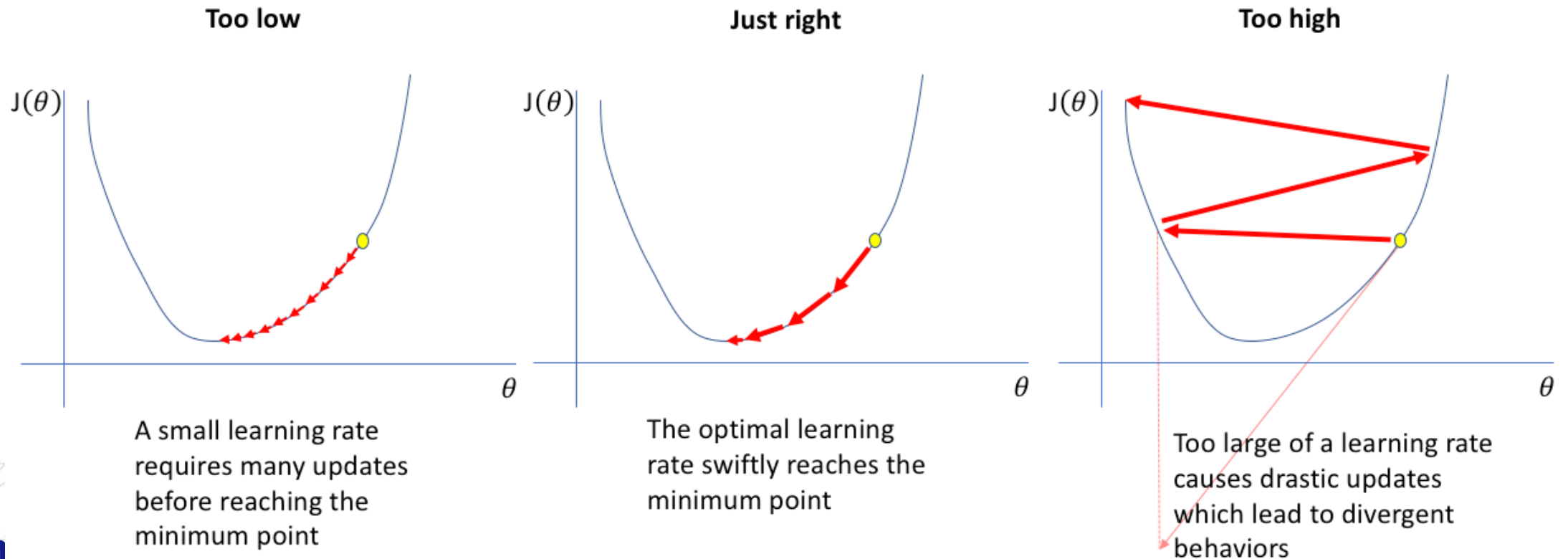
$$\mathbf{x}_{t+1} = \mathbf{x}_t - \nabla f(\mathbf{x}_t)$$



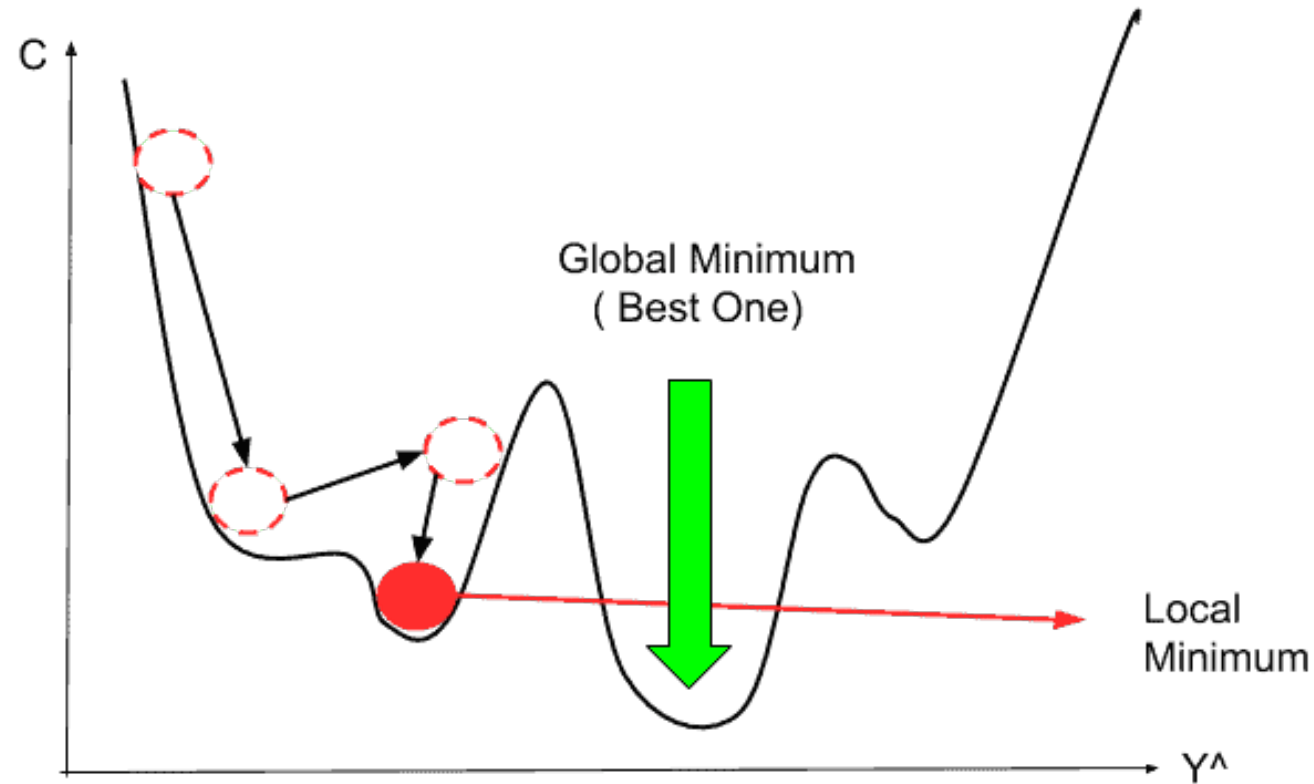
Learning rate

- Need to restrict the size of the steps by shrinking the direction vector using a **learning rate** η , whose value is less than 1:

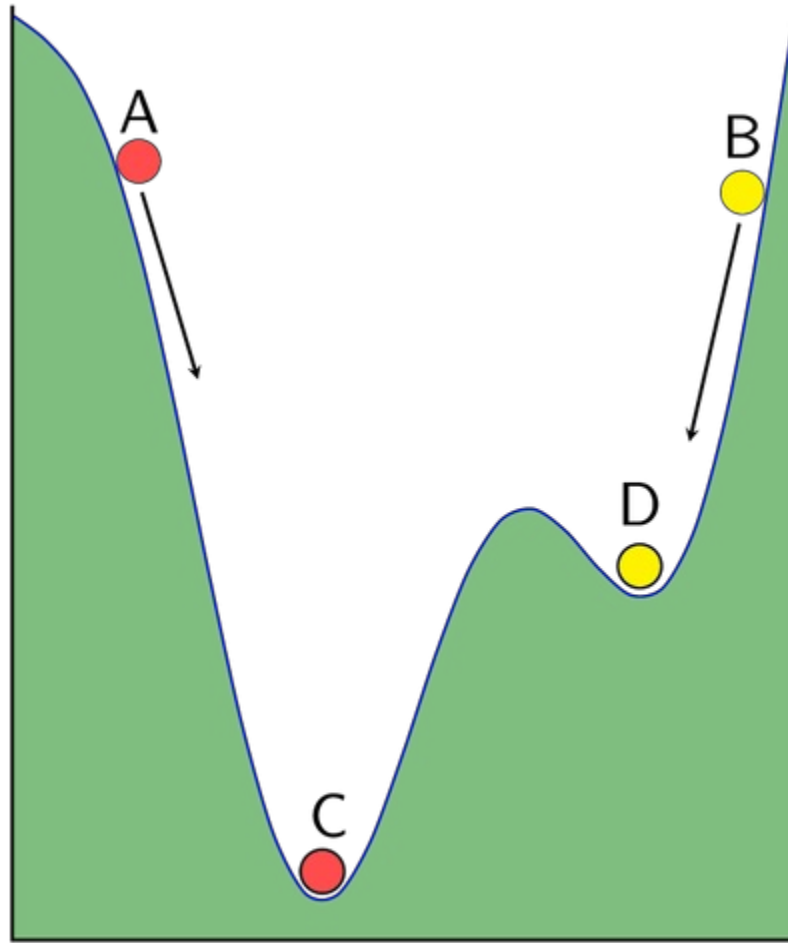
$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$$



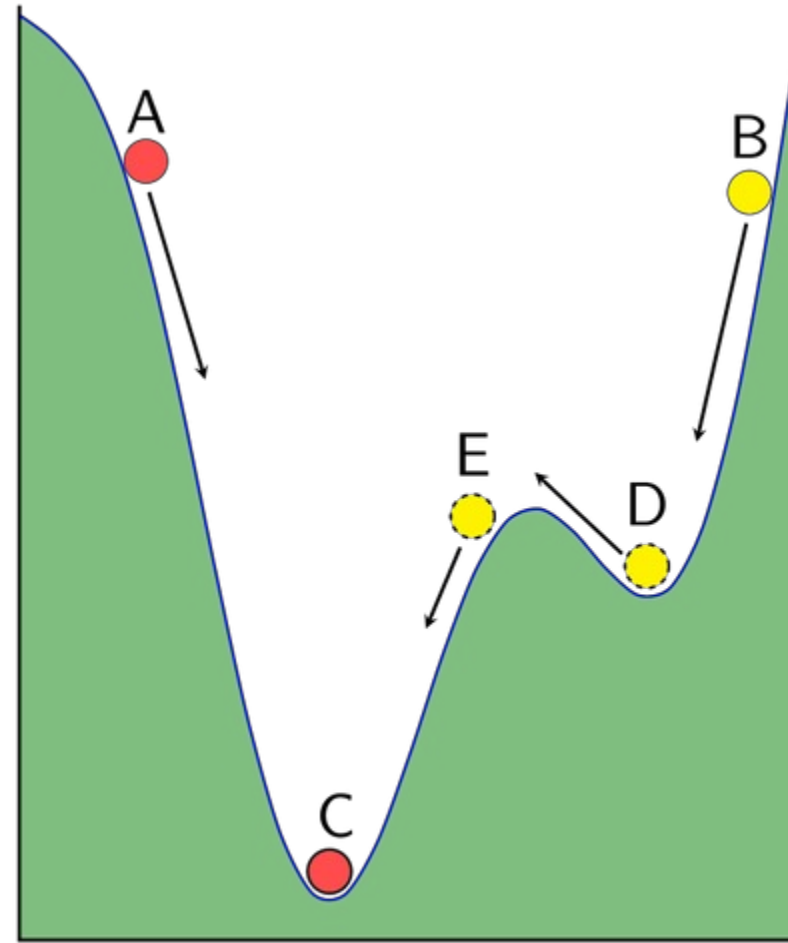
Issues: Starting point (non-linear function)



Momentum



b) GD

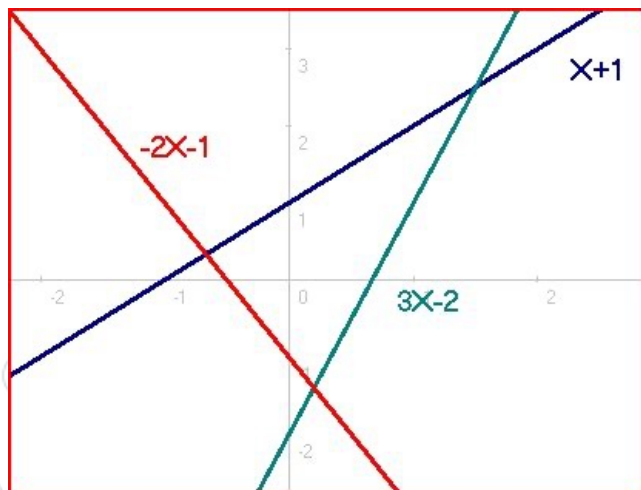


c) GD with momentum

Over-determined systems

◎ **Over-determined systems** have more constraints (equations) than unknown variables.

- **No solutions** satisfying the linear system.
- **Approximate solutions** to minimize a given error.

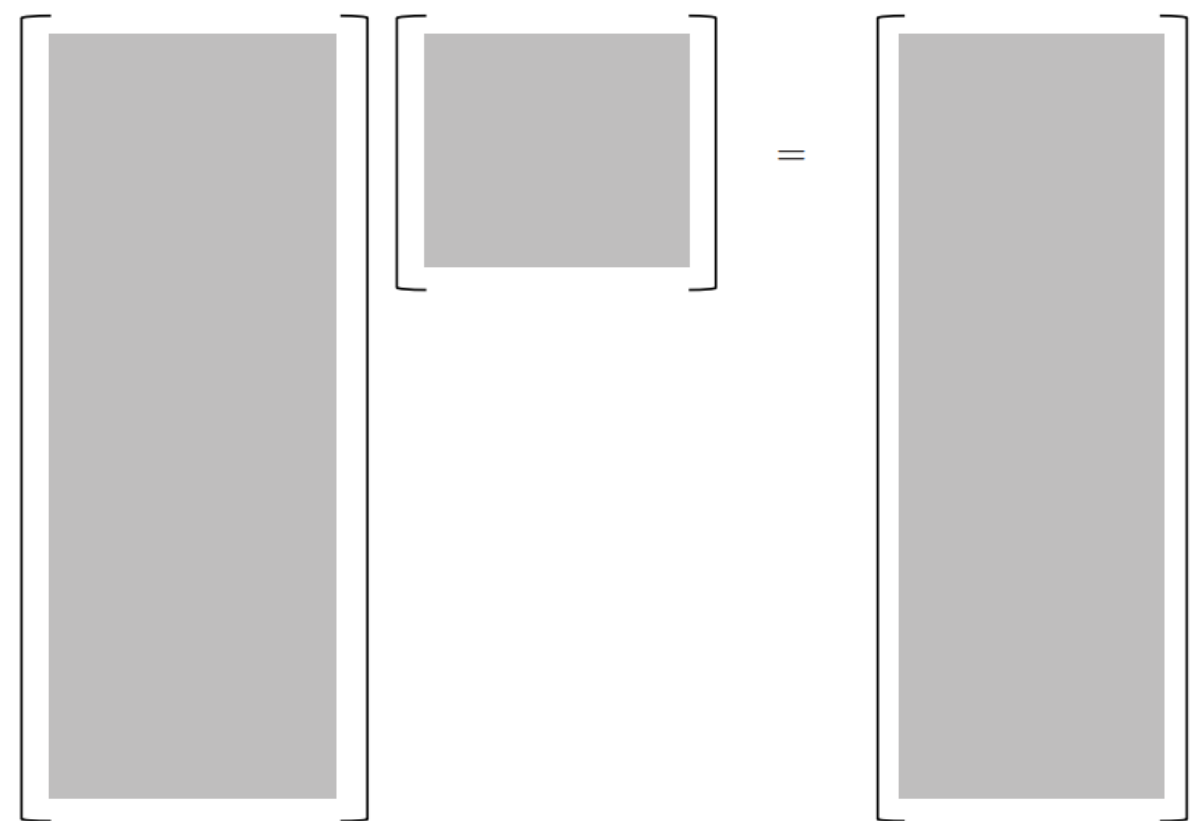


Model terms Loadings Outcomes

A x b

$=$

$=$



Under-Determined Systems

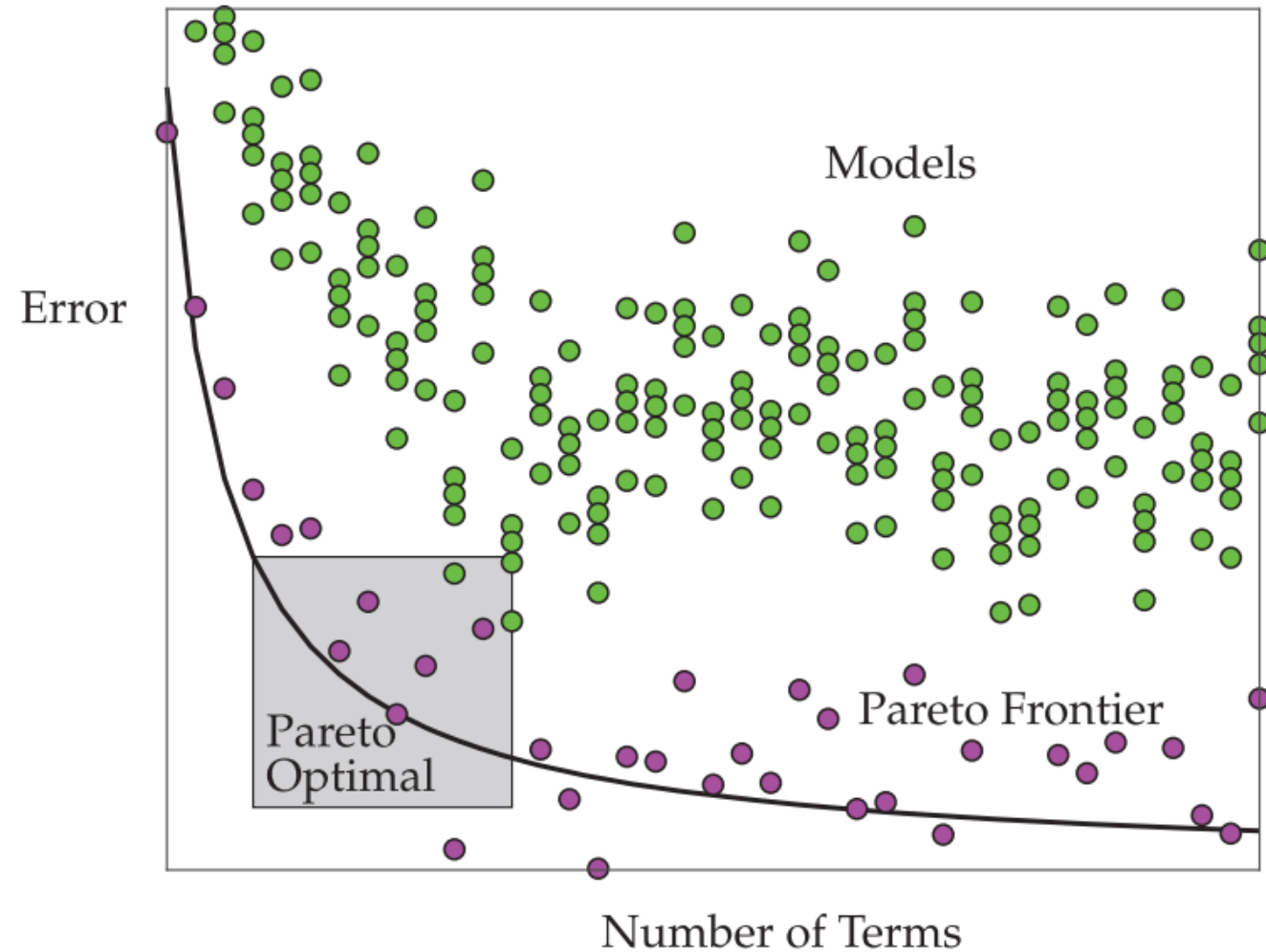
◎ **Under-determined systems** have more unknowns than constraints.

- an infinite number of solutions.
- some choice of constraint must be made.


$$\begin{array}{ccccc} \text{Model terms} & & \text{Loadings} & & \text{Outcomes} \\ A & & x & = & b \\ \left[\begin{array}{c} \text{ } \end{array} \right] & & \left[\begin{array}{c} \text{ } \end{array} \right] & = & \left[\begin{array}{c} \text{ } \end{array} \right] \end{array}$$

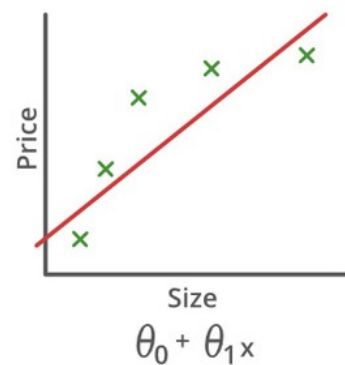
Model Selection

- Model selection is not simply about reducing error, it is about producing a model that has a **high degree of interpretability**, **generalization** and **predictive capabilities**.

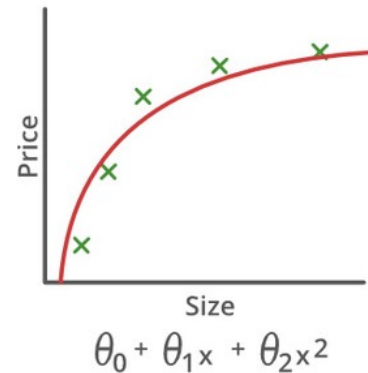


Overfitting

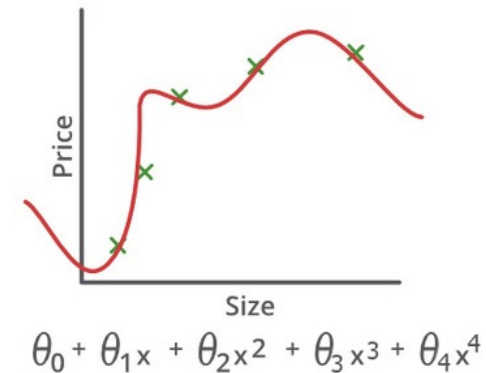
- ◎ The production is too closely to a particular set of data, and may therefore fail to fit to predict future observations reliably.
 - Overfitting does not allow for generalization.



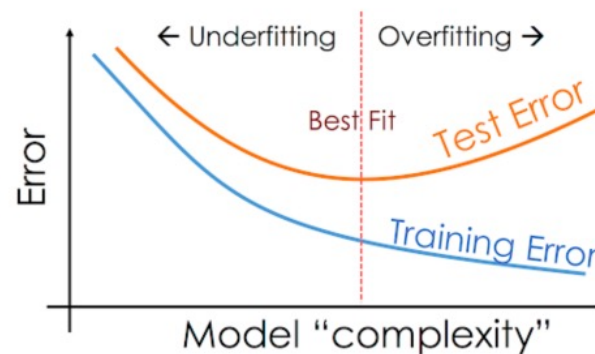
High bias (underfit)



Good fit



High variance
(overfit)





The End